A
Mini Project Report
On

Movie Booking System

Presented by

1. Sachin Choudhary
Roll No.: 118
2. Sudarshan Bhoskar
Roll No.: 111

Computer Science and Engineering
(2026-2027)

Guided By

Ms. M. G. Shelke

(Department of Computer Science and Engineering)

Submitted to



MGM's College of Engineering, Nanded

Under

Dr. Babasaheb Ambedkar Technological University, Lonere

Certificate

This is to certify that the mini project entitled

Movie Booking System

Submitted By

Sachin Choudhary
&
Sudarshan Bhoskar

In satisfactory manner as a partial fulfillment of
TY[CSE-A] in Computer Science and Engineering

To

MGM's College of Engineering, Nanded

Under
Dr. Babasaheb Ambedkar Technological University, Lonere
has been carried out under my guidance,

Ms. M. G. Shelke
Mini Project Guide

Dr. Mrs. A. M. Rajurkar	**Dr. Mrs. G. S. Lathkar**
H.O.D	**Director**
Computer Science & Engineering	MGM's College of Engg., Nanded

1. Abstract

CineBook – Movie Ticket Booking System is a web-based application developed to simplify and manage the movie ticket booking process. The system allows users to view available movies, check theatre information, explore show timings and book movie tickets through a user-friendly interface.

The project is developed using HTML and CSS for the frontend, Python Flask for backend operations, and MySQL as the database. PyMySQL is used to establish communication between the Flask application and the MySQL database.

The database maintains important information related to users, movies, theatres, shows and bookings. It uses relational database concepts such as primary keys, foreign keys and constraints to maintain data consistency and establish relationships between different tables.

The main purpose of CineBook is to provide a simple and organized platform for managing movie information and ticket bookings. This project also demonstrates the practical implementation of Database Management System concepts in a real-world application.

---

2. Problem Statement

In traditional movie ticket booking systems, customers may need to visit theatres physically or depend on manual booking processes to obtain movie tickets. These processes can be time-consuming and may create difficulties in checking movie details, show timings and booking information.

Managing movie details, theatre information, show schedules and customer bookings manually can also become difficult. Maintaining records manually may lead to data duplication, errors and difficulties in retrieving information.

Therefore, there is a need for a computerized system that can organize movie-related information and manage booking records efficiently.

CineBook aims to develop a web-based Movie Ticket Booking System that provides an organized way to manage movies, theatres, show timings, users and ticket bookings using a relational database.

---

3. Objectives of the Project

The main objectives of the CineBook project are:

1. To develop a simple and user-friendly Movie Ticket Booking System.
2. To display movie information in an organized manner.
3. To provide theatre details and available show timings.
4. To allow users to register and log in to the system.
5. To provide a facility for booking movie tickets.
6. To maintain user and booking information in a MySQL database.
7. To establish relationships between database tables using primary keys and foreign keys.
8. To implement SQL operations for inserting, updating, deleting and retrieving records.
9. To demonstrate practical DBMS concepts through a real-world application.
10. To reduce manual work and improve the organization of booking information.

---

4. Scope of the Project

The scope of the CineBook project includes the development of a basic web-based movie ticket booking application.

The system focuses on the following areas:

1. User Management

- User registration and login.
- Storing user details in the database.
- Managing user-related information.

2. Movie Management

- Displaying available movie details.
- Maintaining movie information such as title, genre and duration.
- Retrieving movie records from the database.

3. Theatre Management

- Maintaining theatre information.
- Displaying theatre details to users.

4. Show Management

- Managing movie show schedules.
- Displaying show dates and timings.
- Connecting movies, theatres and shows through database relationships.

5. Booking Management

- Allowing users to book movie tickets.
- Storing booking information in the database.
- Displaying booking history to users.

6. Database Management

- Maintaining relational tables.
- Implementing primary keys and foreign keys.
- Performing SQL queries for database operations.
- Retrieving information using joins, aggregate functions and other SQL operations.

The project is designed as an academic DBMS mini project. Advanced payment gateway integration, real-time seat synchronization and online ticket verification are outside the current scope unless implemented separately.

---

5. Proposed System

The proposed system is a web-based Movie Ticket Booking System named CineBook. It provides a centralized platform for managing movie details, theatre information, show timings, user records and ticket bookings.

The system uses HTML and CSS to create the user interface. Python Flask handles the backend operations and communicates with the MySQL database using PyMySQL.

Users can register or log in, view movie details, check theatre and show information, and perform ticket booking operations through the application.

All important information is stored in a relational database. Different tables are connected through primary keys and foreign keys to maintain data relationships and consistency.

Features of the Proposed System:

- Simple and user-friendly interface.
- Online display of movie information.
- Organized theatre and show details.
- User registration and login.
- Movie ticket booking facility.
- Booking record management.
- Database-based storage and retrieval.
- Efficient execution of SQL queries.
- Reduced dependency on manual record management.

6. Existing System and Its Limitations

Existing System

In a traditional movie ticket booking process, customers may have to visit the theatre counter to check movie availability and book tickets. Movie details, show schedules and customer booking records may be maintained through manual processes or separate systems.

This approach can create difficulties in managing information and retrieving booking records efficiently.

Limitations of the Existing System

1. Time-Consuming Process: Customers may need to visit the theatre physically to book tickets.
2. Manual Record Management: Maintaining customer and booking records manually requires additional effort.
3. Data Duplication: Repeated manual entries may result in duplicate records.
4. Difficulty in Searching: Finding particular movie or booking information can take time.
5. Limited Accessibility: Customers may not be able to access booking information conveniently from different locations.
6. Human Errors: Manual data entry can result in incorrect information.
7. Difficult Record Maintenance: Updating and maintaining large amounts of booking information can be challenging.

The proposed CineBook system attempts to address these limitations by organizing information through a web application and relational database.

---

7. Hardware Requirements

The following hardware configuration is suitable for developing and running the CineBook project.

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Hard Disk	500 GB HDD or 128 GB SSD
Display	Standard Monitor
Keyboard	Standard Keyboard
Mouse	Standard Mouse
Network	Internet connection for development and GitHub

8. Software Requirements

Software	Purpose
Operating System	Windows 10/11
Frontend	HTML
Styling	CSS
Backend	Python
Web Framework	Flask
Database	MySQL
Database Connector	PyMySQL
Code Editor	Visual Studio Code
Version Control	Git
Code Hosting	GitHub
Web Browser	Google Chrome or Microsoft Edge

---

9. Feasibility Study

A feasibility study is performed to determine whether the proposed system can be developed and operated using the available resources.

The feasibility of the CineBook project is explained below.

1. Technical Feasibility

The project is technically feasible because it uses commonly available technologies such as HTML, CSS, Python Flask and MySQL. These technologies can run on a standard computer system and provide the required functionality for a basic movie ticket booking application.

2. Economic Feasibility

The project is economically feasible because the main technologies used, including Python, Flask, MySQL Community Edition, HTML and CSS, are available without purchasing expensive software licences. The project can be developed using an existing computer and internet connection.

3. Operational Feasibility

The system is operationally feasible because it provides a simple and user-friendly interface. Users can easily navigate through movie details, theatre information, show timings and booking-related sections.

4. Schedule Feasibility

The project is suitable for development within an academic project schedule. It can be divided into different stages, including database design, frontend development, backend development, database connectivity, testing and documentation.

Conclusion of Feasibility Study

Based on the technical, economic, operational and schedule analysis, the CineBook project is feasible for implementation as a college-level DBMS mini project.

---

10. System Architecture

The CineBook system follows a three-layer architecture consisting of the Presentation Layer, Application Layer and Database Layer.

1. Presentation Layer (Frontend)

The presentation layer is the user interface of the CineBook application. It is developed using HTML and CSS.

Responsibilities:

- Displays the home page.
- Shows movie and theatre information.
- Displays show timings.
- Provides login and registration interfaces.
- Provides booking-related interfaces.
- Accepts user input.

2. Application Layer (Backend)

The application layer is developed using Python Flask. It processes user requests and connects the frontend with the database through PyMySQL.

Responsibilities:

- Handles application routes.
- Processes user registration and login requests.
- Retrieves movie and theatre information.
- Processes booking-related operations.
- Executes SQL queries.
- Sends database results to the frontend.

3. Database Layer (MySQL)

The database layer stores and manages all project-related information.

Main database tables:

- Users
- Movies
- Theatres
- Shows
- Bookings

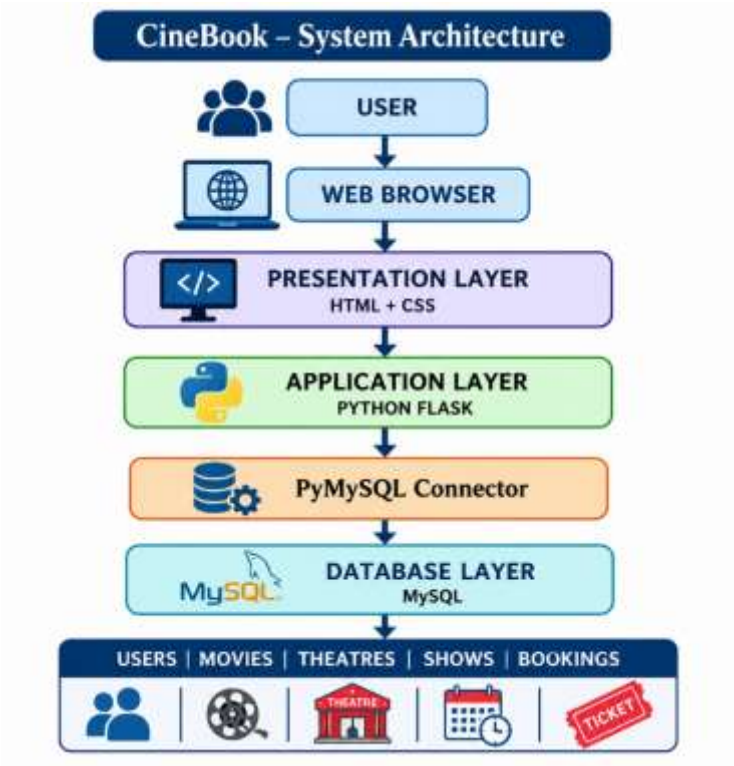
Responsibilities:

- Stores user details.
- Maintains movie and theatre records.
- Stores show information.
- Maintains booking records.
- Maintains relationships using primary keys and foreign keys.
- Retrieves information when requested by the backend.

Working of System Architecture

The working process of the CineBook system is as follows:

1. The user interacts with the CineBook website through a web browser.
2. The frontend sends the user's request to the Flask backend.
3. Flask processes the request.
4. PyMySQL establishes communication with the MySQL database.
5. MySQL executes the required SQL query.
6. The database returns the requested information.
7. Flask processes the result and sends it to the frontend.
8. The frontend displays the result to the user.



The architecture separates the user interface, application logic and database operations, making the system easier to understand, maintain and modify.

---

11. System Flow

START	User opens the CineBook website
1	Register or log in
2	Flask verifies the user and creates a session
3	Website displays movies
4	User selects a movie and views its shows
5	User selects a show and enters ticket quantity
6	System calculates total price and stores the booking
7	User can view booking history or log out
END	Session ends / user logs out

---

12. Database Design

Database design is an important part of the CineBook – Movie Ticket Booking System. The system uses a relational database management system (RDBMS), MySQL, to store, organize and manage information related to users, movies, theatres, shows and bookings.

The database is designed to maintain relationships between different entities and provide efficient storage and retrieval of information.

The database consists of the following main tables:

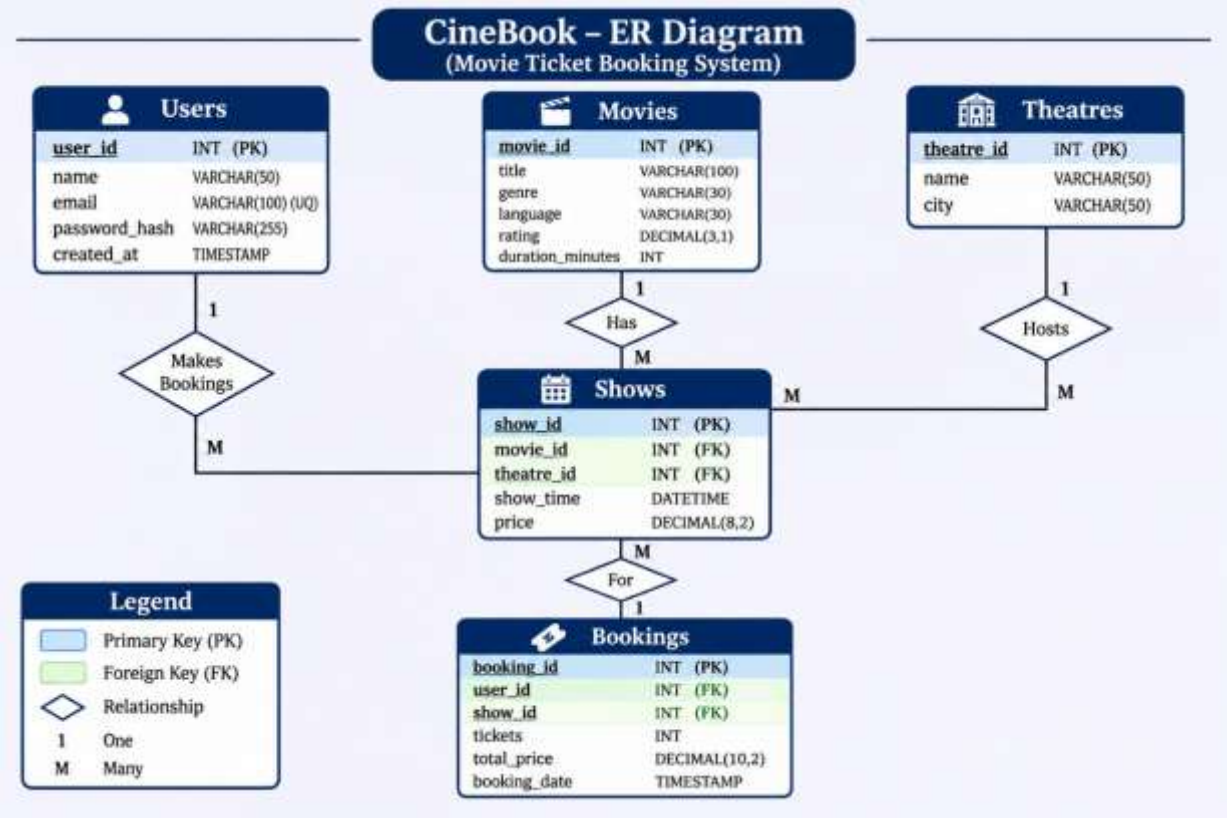
1. Users: Stores information about registered users.
2. Movies: Stores details of movies available in the system.
3. Theatres: Stores information about theatres.
4. Shows: Stores movie show details, including show timings and related information.
5. Bookings: Stores ticket booking records made by users.

These tables are connected using primary keys and foreign keys. This helps maintain relationships between records and reduces unnecessary data duplication.

Advantages of Database Design:

- Organized storage of information.
- Easy retrieval and management of records.
- Maintains relationships between tables.
- Reduces data duplication.
- Improves data consistency.
- Supports efficient execution of SQL queries.

13. ER Diagram (Entity Relationship Diagram)



An Entity Relationship (ER) Diagram is a graphical representation of the database structure. It shows the different entities, their attributes and the relationships between them.

In the CineBook project, the ER Diagram represents the relationship between Users, Movies, Theatres, Shows and Bookings.

Description of Entities:

- Users: Represents customers who register and use the application.
- Movies: Represents the movies displayed in the application.
- Theatres: Represents theatres where movies are shown.
- Shows: Represents scheduled screenings of movies in theatres.
- Bookings: Represents ticket bookings made by registered users.

Relationships Between Entities:

1. A user can make multiple bookings.
2. A movie can have multiple shows.
3. A theatre can organize multiple shows.
4. A show can be associated with multiple bookings.
5. Each booking is associated with a particular user and show.

The ER Diagram helps us understand the overall database structure and the connections between different entities.

---

14. Relational Schema

A relational schema represents the logical structure of a relational database. It defines the tables, their attributes, primary keys and foreign keys.

The proposed relational schema of CineBook is:

```
USERS (
    user_id PRIMARY KEY,
    name,
    email,
    password
)

MOVIES (
    movie_id PRIMARY KEY,
    title,
    genre,
    duration
)

THEATRES (
    theatre_id PRIMARY KEY,
    theatre_name,
    location
)

SHOWS (
    show_id PRIMARY KEY,
    movie_id FOREIGN KEY,
    theatre_id FOREIGN KEY,
    show_date,
    show_time
)

BOOKINGS (
    booking_id PRIMARY KEY,
    user_id FOREIGN KEY,
    show_id FOREIGN KEY,
    booking_date,
    total_seats,
    total_amount
)
```

Explanation:

- Each table has a primary key to uniquely identify its records.
- The `movie_id` in the Shows table refers to the Movies table.
- The `theatre_id` in the Shows table refers to the Theatres table.
- The `user_id` in the Bookings table refers to the Users table.

- The `show_id` in the Bookings table refers to the Shows table.

This relational structure helps establish connections between different tables and supports efficient data retrieval.

Note: The above is an illustrative schema. Match the column names and data types with your actual schema.sql file before submitting the report.

---

15. Database Tables and Their Description

The CineBook database contains different tables to manage information related to the movie ticket booking process.

15.1 Users Table

Purpose: Stores the personal and login information of registered users.

Attribute	Description
user_id	Unique identification number of the user
name	Name of the user
email	Email address of the user
password	User's login password or password hash

15.2 Movies Table

Purpose: Stores information about movies available in the application.

Attribute	Description
movie_id	Unique identification number of the movie
title	Name of the movie
genre	Category of the movie
duration	Duration of the movie

15.3 Theatres Table

Purpose: Stores information about theatres where movies are screened.

Attribute	Description
theatre_id	Unique identification number of the theatre
theatre_name	Name of the theatre
location	Location of the theatre

15.4 Shows Table

Purpose: Stores information about movie screenings, including movie, theatre and show schedule.

Attribute	Description
show_id	Unique identification number of the show
movie_id	Identifies the movie
theatre_id	Identifies the theatre
show_date	Date of the show
show_time	Time of the show

15.5 Bookings Table

Purpose: Stores information about tickets booked by users.

Attribute	Description
booking_id	Unique identification number of the booking
user_id	Identifies the user who made the booking
show_id	Identifies the selected show
booking_date	Date of booking
total_seats	Number of seats booked
total_amount	Total booking amount

Important: These are example attributes for documentation. If your actual database contains different fields, use those fields instead. Do not add columns that are not implemented.

---

16. Primary Keys and Foreign Keys

Keys are important components of a relational database. They help uniquely identify records and establish relationships between different tables.

16.1 Primary Key

A Primary Key is a column or combination of columns that uniquely identifies each record in a database table.

Characteristics:

- Each record must have a unique primary key.
- A primary key cannot contain NULL values.
- A table can have only one primary key constraint.
- It helps identify records uniquely.

Primary Keys in CineBook:

Table Name	Primary Key
Users	user_id
Movies	movie_id
Theatres	theatre_id
Shows	show_id
Bookings	booking_id

16.2 Foreign Key

A Foreign Key is a column in one table that refers to a primary key in another table. It is used to establish relationships between tables.

Foreign Keys in CineBook:

Table	Foreign Key	References
Shows	movie_id	Movies(movie_id)
Shows	theatre_id	Theatres(theatre_id)
Bookings	user_id	Users(user_id)
Bookings	show_id	Shows(show_id)

Example SQL Query:

```
CREATE TABLE Shows (
    show_id INT PRIMARY KEY,
    movie_id INT,
    theatre_id INT,
    FOREIGN KEY (movie_id)
        REFERENCES Movies(movie_id),
    FOREIGN KEY (theatre_id)
        REFERENCES Theatres(theatre_id)
);
```

Importance of Primary Keys and Foreign Keys:

- Maintain data integrity.
- Prevent invalid relationships between records.
- Connect related database tables.
- Help retrieve related information using SQL JOIN operations.

17. Database Constraints

Database constraints are rules applied to table columns to maintain the accuracy, validity and consistency of data.

CineBook uses relational database constraints to ensure that valid information is stored in the database.

17.1 NOT NULL Constraint

The NOT NULL constraint ensures that a column cannot store NULL values.

Example:

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
```

Here, the user's name cannot be left NULL.

17.2 UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column are different.

Example:

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    email VARCHAR(100) UNIQUE
);
```

Here, duplicate email values are not allowed.

17.3 PRIMARY KEY Constraint

The PRIMARY KEY constraint uniquely identifies every record in a table.

Example:

```
CREATE TABLE Movies (
    movie_id INT PRIMARY KEY,
    title VARCHAR(100)
);
```

Here, `movie_id` uniquely identifies each movie.

17.4 FOREIGN KEY Constraint

The FOREIGN KEY constraint establishes a relationship between two tables.

Example:

```
FOREIGN KEY (user_id)
REFERENCES Users(user_id)
```

Here, the user_id in the referencing table must refer to an existing user record, subject to the foreign key rules.

17.5 DEFAULT Constraint

The DEFAULT constraint automatically assigns a predefined value to a column when no value is supplied for it.

Example:

```
CREATE TABLE Bookings (
    booking_id INT PRIMARY KEY,
    status VARCHAR(30) DEFAULT 'Pending'
);
```

Here, the default booking status is set to Pending when no other value is provided.

Importance of Database Constraints

- Maintain data accuracy.
- Prevent invalid data entry.
- Avoid duplicate values where uniqueness is required.
- Maintain relationships between tables.
- Improve database consistency and reliability.

---

18. Implementation

The implementation phase involves developing the CineBook – Movie Ticket Booking System using HTML, CSS, Python Flask and MySQL. The system is divided into three main parts: frontend development, backend development and database implementation.

The frontend provides an interface for users to interact with the application, while the backend handles application logic and database communication. MySQL is used to store and manage all the project-related information.

18.1 Frontend Implementation

The frontend of CineBook is developed using HTML and CSS.

HTML (HyperText Markup Language):

HTML is used to create the structure of the website. It provides different sections such as:

- Home section
- Movies section
- Theatres section
- Shows section
- Login and Registration section
- My Bookings section
- About section

CSS (Cascading Style Sheets):

CSS is used to design and improve the appearance of the website.

It provides:

- Background colours and text styling.
- Navigation bar design.
- Movie cards and content layouts.
- Buttons and form styling.
- Tables and section spacing.
- Responsive layout using flexible elements.

18.2 Backend Implementation

The backend is developed using Python Flask, a lightweight web framework.

Flask handles requests received from the frontend and performs the required application operations.

Main responsibilities include:

- Handling website routes.
- Processing user registration and login.
- Retrieving movie and theatre information.
- Managing show information.
- Processing ticket booking requests.
- Displaying booking information.

18.3 Database Implementation

MySQL is used as the database management system for CineBook.

The database is named `cinebook` and contains five main tables:

1. Users
2. Movies
3. Theatres
4. Shows
5. Bookings

The database stores user details, movie information, theatre details, show schedules and booking records.

Primary keys and foreign keys are used to establish relationships between tables and maintain data integrity.

18.4 Database Connectivity

PyMySQL is used to connect the Python Flask application with the MySQL database.

The connection allows the backend to execute SQL queries and perform database operations.

The basic working process is:

1. The user performs an operation on the website.
2. The request is sent to the Flask backend.
3. Flask processes the request.
4. PyMySQL communicates with the MySQL database.
5. MySQL executes the required query.
6. The result is returned to Flask.
7. Flask sends the response to the frontend.

18.5 SQL Operations Implemented

The following DBMS operations are demonstrated in the CineBook project:

SQL Concept	Purpose
CREATE	Creates database tables
ALTER	Modifies the Movies table
DROP	Removes an existing database
INSERT	Adds records into tables
UPDATE	Modifies existing records
DELETE	Removes records
SELECT	Retrieves database information
JOIN	Combines information from multiple tables
GROUP BY	Groups records based on a column
ORDER BY	Sorts query results
Aggregate Functions	Performs COUNT, SUM, AVG, MIN and MAX
Subqueries	Executes queries inside other queries
VIEW	Creates BookingDetails view
INDEX	Improves searching and sorting by show_time

These operations demonstrate the practical implementation of DBMS concepts in a real-world application.

19. Project File Structure

The CineBook project is organized into separate files to make development, maintenance and understanding easier.

The project contains frontend files, backend code and the database script.

19.1 Folder Structure

```
CineBook/
|
```

```
├── app.py
├── index.html
├── style.css
└── schema.sql
```

19.2 Description of Files

1. app.py – Backend File

- Contains the Python Flask application.
- Defines application routes.
- Handles user requests.
- Connects Flask with MySQL using PyMySQL.
- Performs database operations.

2. index.html – Frontend File

- Contains the structure of the website.
- Displays different sections of CineBook.
- Provides forms, buttons, navigation and content.
- Displays information received from the backend.

3. style.css – Styling File

- Defines the visual appearance of the website.
- Controls colours, fonts and layouts.
- Styles navigation bars, buttons, cards and forms.
- Improves the overall user interface.

4. schema.sql – Database File

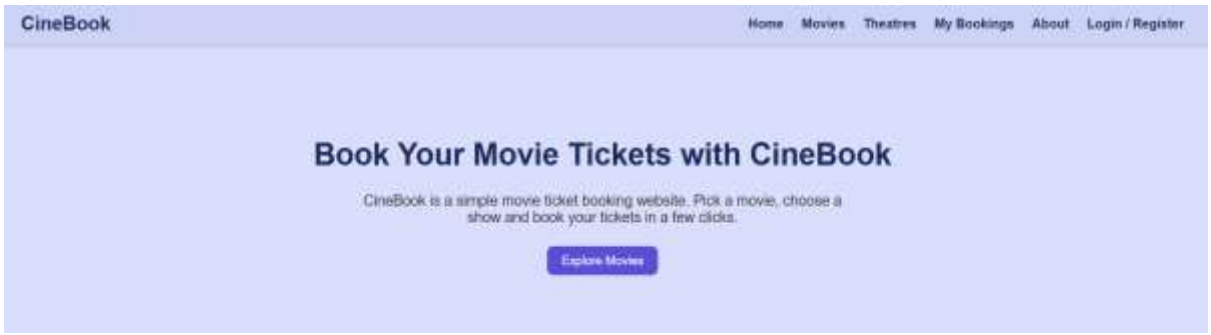
- Creates the CineBook database.
- Creates the required database tables.
- Defines primary keys and foreign keys.
- Applies database constraints.
- Inserts sample movie and theatre records.
- Creates the BookingDetails view.
- Creates an index on the show_time column.
- Contains SQL demonstration queries.

19.3 Importance of Project File Structure

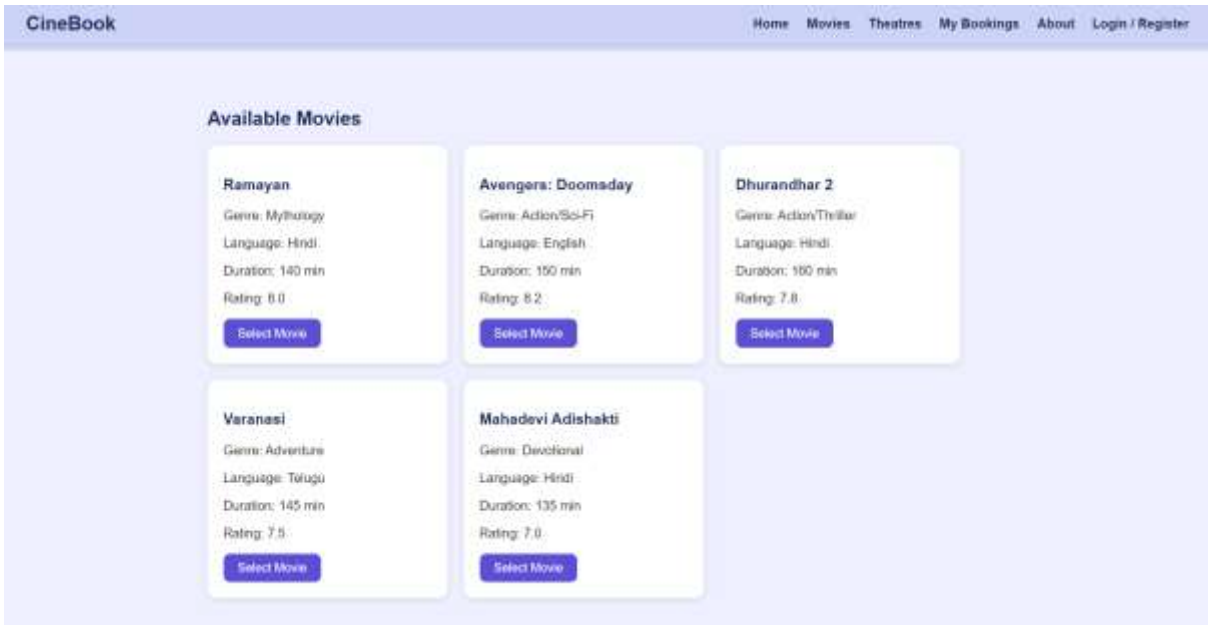
- Keeps project files organized.
- Separates frontend, backend and database code.
- Makes debugging easier.
- Helps in maintaining and modifying the application.
- Makes the project easier for team members to understand and manage.

20. OUTPUT SCREENSHOTS

Home page



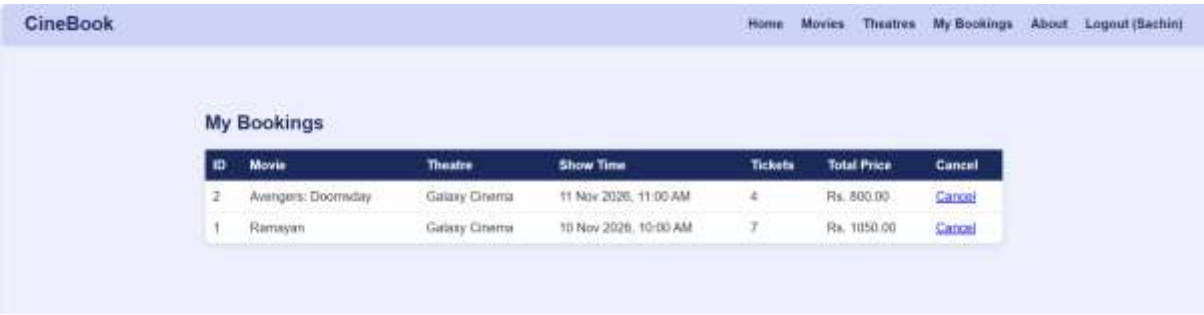
Movies Page

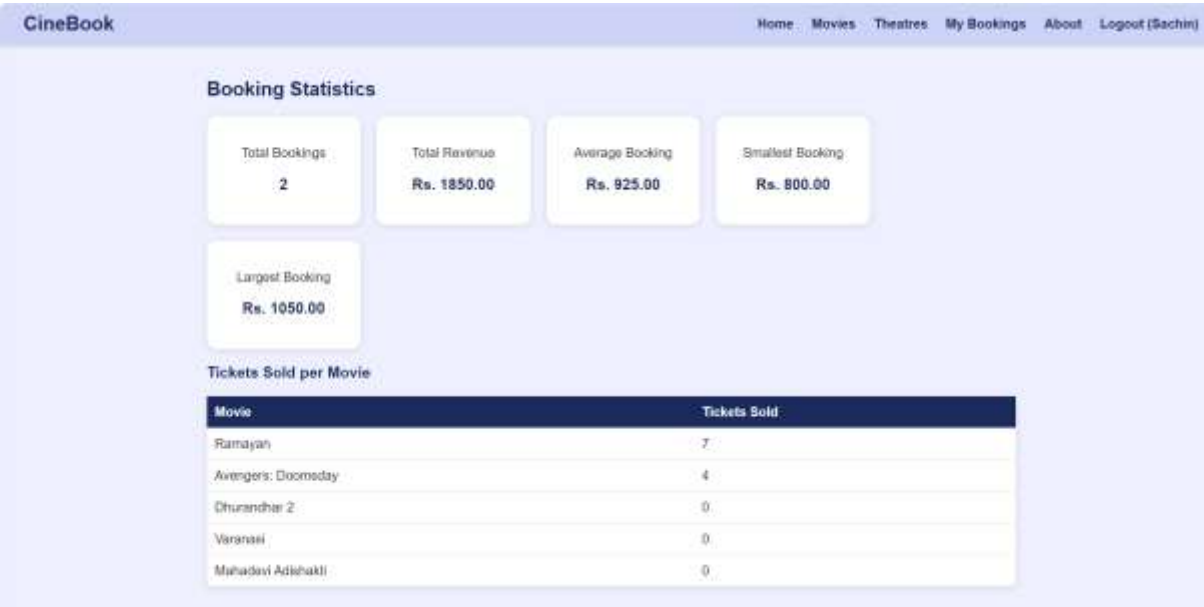


Theaters



My Bookings





---

21. Advantages

The CineBook – Movie Ticket Booking System offers several advantages:

1. Easy to Use: Provides a simple and user-friendly interface for browsing movies and booking tickets.
2. Time Saving: Reduces the time required to visit a theatre and book movie tickets manually.
3. Centralized Database: Stores user, movie, theatre, show and booking information in a single MySQL database.
4. Easy Data Management: Allows movie details, show timings and booking records to be managed using SQL queries.
5. Data Consistency: Primary keys, foreign keys and database constraints help maintain data accuracy and consistency.
6. Quick Access to Information: Users can easily view available movies, theatre details and show timings.

7. Booking Management: Stores booking details systematically and allows users to view their previous bookings.
8. Secure Password Storage: Uses password hashing to avoid storing user passwords in plain text.
9. Reduced Manual Work: Automates basic operations such as user registration, login and ticket booking.
10. Easy Maintenance: The modular use of HTML, CSS, Python Flask and MySQL makes the system easier to maintain and improve.

---

22. Limitations

Although CineBook provides a simple and useful movie ticket booking system, it has some limitations:

1. Limited Movie Collection: The system currently contains only a few sample movies.
2. Single Theatre: The current database includes only one theatre, Galaxy Cinema, located in Nanded.
3. Sample Data: Movie ratings, details and show prices are sample data.
4. Basic User Interface: The website provides basic movie browsing and booking functionality.
5. No Online Payment Gateway: The system does not currently support real online payment transactions.
6. Limited Seat Management: Advanced seat selection and real-time seat availability management are not included in the current implementation.
7. Internet and Server Dependency: The website requires the Flask application and MySQL database server to be running.

---

23. Future Scope

CineBook can be improved and extended with additional features in the future:

1. Multiple Theatre Support: Add multiple theatres across different cities.
2. Seat Selection System: Implement an interactive seat matrix where users can select their preferred seats.
3. Online Payment Integration: Integrate payment gateways for secure online transactions.
4. Movie Search and Filters: Allow users to search movies by name, genre, language and rating.
5. Booking Confirmation: Generate digital tickets with unique booking IDs and QR codes.
6. Admin Dashboard: Provide an admin panel to manage movies, theatres, shows and bookings.
7. Email Notifications: Send booking confirmations and cancellation notifications through email.

8. Mobile-Friendly Design: Improve the interface for better accessibility on smartphones and tablets.

---

24. Conclusion

CineBook – Movie Ticket Booking System is a DBMS mini project developed using HTML, CSS, Python Flask and MySQL.

The project demonstrates the practical implementation of database management concepts such as tables, primary keys, foreign keys, relationships, constraints, SQL queries, joins, views and indexes.

Through this project, we gained practical knowledge of connecting a web application with a MySQL database using Python Flask. We also understood how user information, movie details, theatre information, show timings and booking records can be managed systematically.

Overall, CineBook provides a basic platform for understanding the working of an online movie ticket booking system and offers opportunities for future improvements and additional features.